

ORRAM: An OpenROAD-Integrated RAM Generator Using Standard Cells

Brayden Louie^{*1}, Thinh P. Nguyen^{*1}, Matt Liberty², Austin Rovinski¹

¹New York University, ²Precision Innovations, Inc.

{brayden.louie,pn2409,rovinski}@nyu.edu, mliberty@precisioninno.com

1 Introduction

Memory inference remains a significant challenge in turnkey ASIC design flows. Inferring flip-flops from RTL can create thousands of densely interconnected instances which dramatically slow down design flows and impede performance. Memory compilers address this issue, although they are third-party tools which are often PDK-specific and may require specialized cells not in the base PDK.

To address these shortcomings, we present ORRAM, a standard-cell-based memory generator built as a native module within OpenROAD. Given a standard cell library, ORRAM produces a fully placed and routed RAM block requiring no custom bitcells or external tooling, with timing verification via OpenSTA rather than SPICE simulation. ORRAM supports arbitrary word sizes, word counts, mask granularities, multi-port read configurations, column muxing, latch-based storage, and automatic PDK-agnostic cell selection, making it compatible with most standard cell libraries including sky130hd and NanGate45. Evaluated on SkyWater 130nm, ORRAM matches the bit density of historical DFFRAM results while offering a significantly expanded feature set. The source code is available as part of the OpenROAD project [3].

2 Related Work

Several open-source alternatives exist for generating on-chip memories, however they each have shortcomings for designers seeking high productivity with low barrier to entry. The primary option for digital designers is to declare the memory in RTL, which will be inferred as flip-flops during synthesis and then placed and routed with other logic. Open-source tools like Yosys [6] and OpenROAD [1] handle this methodology; however, this technique can incur a high power/performance/area (PPA) penalty due to synthesis algorithms which do not infer buses and generic placement algorithms that reduce logic utilization. Further, runtime of the overall digital flow is impacted by inserting thousands – possibly millions – of cells into the netlist and slowing down optimization algorithms.

In contrast, memory compilers can alleviate many of the PPA and runtime issues by generating the memory as a block. OpenRAM [2] is an open-source SRAM compiler that constructs high-density SRAM arrays; however, it requires designing custom cells like 6T bitcells and sense amplifiers for each PDK being targeted. Further, it requires expensive SPICE simulations for power and timing info.

DFFRAM [5] instead takes a standard-cell-based approach, constructing RAM arrays from flip-flops/latches and associated logic without requiring custom bitcell design. Previous versions achieved high bit densities through a custom placer. However, DFFRAM now only generates netlists and instead relies on LibreLane [4] for placement and routing, resulting in significantly lower densities.

ORRAM addresses these limitations by taking a fully integrated approach to standard cell memory generation within OpenROAD.

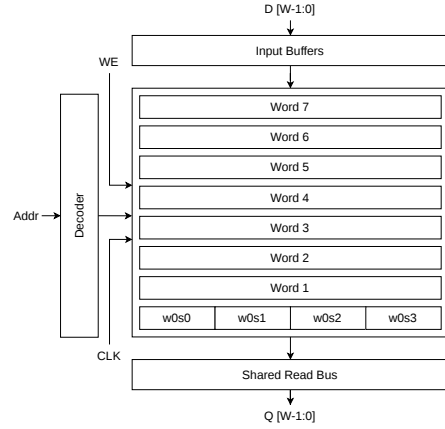


Figure 1: ORRAM Top-Level Architecture

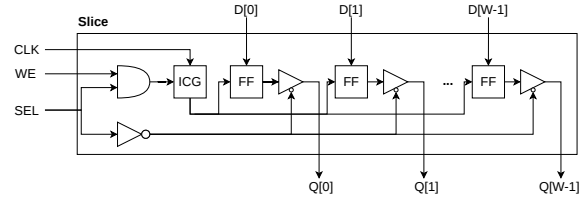


Figure 2: ORRAM Word Slice Architecture

Rather than operating as a standalone compiler, ORRAM [3] generates a fully placed and routed memory block as a native OpenROAD module and performs timing verification via OpenSTA instead of SPICE simulation.

3 Features and Implementation

RAM Structure. ORRAM’s default memory architecture consists of an input buffer stage, a word decoder, and a storage array (Fig. 1). Input buffers drive data signals into the high-fanout core array, while word selection is handled by a decoder. One decoder is generated per RAM port, generating per-word write enable and per-port read select signals. The array is composed of flip-flops/latches, with tristate cells gating each word’s output onto a shared read bus.

Column muxing. Column muxing enables changing the aspect ratio of a RAM by placing multiple words per physical row. This is useful for typical memories which have much larger word counts than word sizes. ORRAM supports a `column_mux_ratio` of {1, 2, 4}, which corresponds to the number of words per physical row. For each physical row, words are interleaved such that each bit is adjacent as shown in Fig. 4. A row of multiplexers is created at the top of the array using AOI22 bit cells, which are generally available

^{*} denotes equal author effort

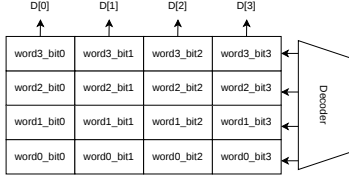


Figure 3: 4x4 RAM, column_mux_ratio=1

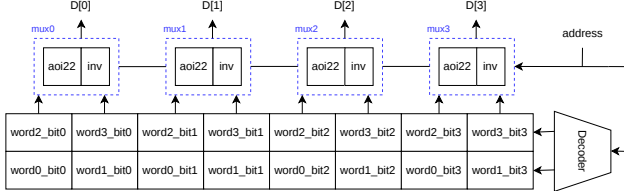


Figure 4: 4x4 RAM, column_mux_ratio=2

in any standard cell library. The lowest $\log_2(\text{column_mux_ratio})$ bits of the address select the word within a row, and the remaining upper bits select the physical row.

Latch memory support. Latch-based memories have considerable advantages over flip-flop-based memories, offering a compact, faster, and more power-efficient alternative. ORRAM offers the option to use latches as storage cells through a two-phase latch setup. When enabled, a row of negative latches is inserted between the input buffer and the storage rows, which now consist of positive latches. During the low clock phase, the negative latch in each column captures and holds the input data; on the rising edge, the positive latches in the cells below capture this data, thus having identical functionality to a positive flip-flop. With column muxing enabled, the negative latch position within each bit group is adjusted to maintain balanced fanout across all storage cells below.

Behavioral Model. ORRAM supports generation of a cycle-accurate, PDK-agnostic behavioral model for all configurations. This behavioral model can be used in RTL simulations for significantly faster simulation than simulating directly with the netlist.

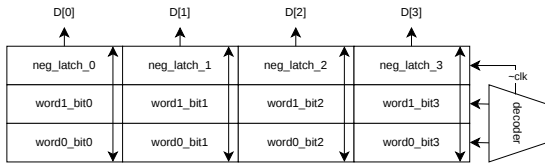


Figure 5: 4x4 latch-based RAM Layout

4 Results and Takeaways

We evaluated ORRAM on sky130hd across a range of memory sizes from 512 bytes to 8 KB, compared to other current open-source alternatives. All configurations target 64-bit word reads with byte-level write masking (Table 1).

ORRAM's DFF-based memory achieves bit densities of approximately 28,000 bits/mm² across all tested sizes, representing roughly 2× the density of the current DFFRAM project, which achieves approximately 14,000–15,000 bits/mm² across the same configurations. This gap stems from DFFRAM relying on LibreLane for placement and routing using generic algorithms, whereas ORRAM

Table 1: Bit density (bits/mm²) across memory generators on SkyWater 130nm.¹

Size	ORRAM Latch ²	ORRAM DFF ²	DFFRAM (historical) ³	DFFRAM (current)	Open- RAM ⁴	RTL+ ORFS ⁵
512 B	31,795	27,734	26,557	14,691	38,299	15,196
1 KB	31,552	28,168	26,027	14,951	77,499	15,250
2 KB	31,977	28,339	26,297	14,817	105,767	15,639
4 KB	32,284	28,586	26,196	14,948	120,240	15,566
8 KB	32,508	28,677	26,229	14,879	148,413	15,649

¹ Configured with 64-bit words and byte-level masking.

² ORRAM results use configurations yielding the highest bit density

³ Historical results reported from the DFFRAM repository; exact configurations are unverifiable as the custom placer has since been removed.

⁴ Dimensions reported are raw compiler output, OpenRAM macros are typically wrapped before being used in automated PnR flows.

⁵ Configured for 60% core utilization.

uses OpenROAD's API to create a custom placer and enable much higher placement regularity and density. Notably, historical DFFRAM results using its deprecated custom placer achieved approximately 26,000–26,500 bits/mm², which ORRAM's DFF-based approach matches while offering a significantly expanded feature set. By comparison, the behavioral RTL approach that uses OpenROAD Flow Scripts (ORFS) achieves approximately 15,000–15,600 bits/mm², and additionally incurs the overhead of executing the full RTL-to-GDS flow including synthesis, clock tree synthesis, and general-purpose placement and routing. OpenRAM achieves higher peak densities at larger sizes (up to ~148,000 bits/mm² at 8 KB) due to its custom 6T bitcell layouts, but comes with the need for hand-designed cells for each target PDK and cumbersome SPICE simulations to measure timing.

ORRAM's latch-based memory provides a 15% density improvement over its DFF counterpart, achieving 31,000–32,000 bits/mm² across tested sizes due to the smaller footprint of latch cells relative to flip-flops, with the trade-off of additional timing analysis complexity due to the two-phase capture behavior. For designs where latch-based timing closure is acceptable, this configuration offers the highest density of any standard-cell-based approach evaluated.

Beyond density, ORRAM provides several capabilities absent from the current DFFRAM project: multi-port read/write configurations, configurable column muxing for aspect ratio control, and native integration within OpenROAD and OpenROAD Flow Scripts. Combined, these features make ORRAM a drop-in memory generation solution for open-source ASIC design without requiring custom bitcell libraries or an external flow setup.

5 Future Work

While ORRAM already offers many features to support faster design space exploration and even tapeouts, we are still working on several features to improve usability and performance including: 1) tightly integrating OpenSTA to perform timing and power-based optimization on RAM configurations, 2) banking support to help support even larger-scale RAMs, and 3) automatically generating RAMs from the memory inference step during synthesis.

References

- [1] Tutu Ajayi, David Blaauw, Tuck-Boon Chan, Chung-Kuan Cheng, Vidya A. Chhabria, David K. Choo, Matteo Coltella, Sorin Dobre, Ronald G. Dreslinski, Mateus Fogaça, Soheil Hashemi, Abdelrahman Hosny, Andrew B. Kahng, Min-soo Kim, Jiajia Li, Zhaoxin Liang, Uday Mallappa, Paul Penzes, Geraldo Pradipta,

- Sherief Reda, Austin Rovinski, Kambiz Samadi, Sachin S. Sapatnekar, Lawrence Saul, Carl Sechen, Vaishnav Srinivas, William Swartz, Dennis Sylvester, David Urquhart, Lutong Wang, Mingyu Woo, and Bangqi Xu. 2019. OpenROAD: Toward a Self-Driving, Open-Source Digital Layout Implementation Tool Chain. In *Proc. GOMATECH*.
- [2] Matthew R. Guthaus, James E. Stine, Samira Ataei, Brian Chen, Bin Wu, and Mehmet Sarwar. 2016. OpenRAM: An Open-Source Memory Compiler. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. <https://doi.org/10.1145/2966986.2980098>
- [3] Brayden Louie, Thinh P. Nguyen, Matt Liberty, and Austin Rovinski. 2026. ORRAM. <https://github.com/The-OpenROAD-Project/OpenROAD/tree/master/src/ram>
- [4] Mohamed Shalan and Tim Edwards. 2020. Building OpenLANE: A 130nm OpenROAD-based Tapeout-Proven Flow: Invited Paper. In *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–6.
- [5] Mohamed Shalan, Mohamed Gaber, Manar Abdelatty, Ahmed Nofal, Hany Moussa, Ramez Moussa, and Efabless Corporation. 2021. DFFRAM: Standard-Cell-Based Memory Compiler. <https://github.com/AUCOHL/DFFRAM>.
- [6] Claire Wolf. [n. d.]. Yosys Open SYNthesis Suite. <https://yosyshq.net/yosys/>.